

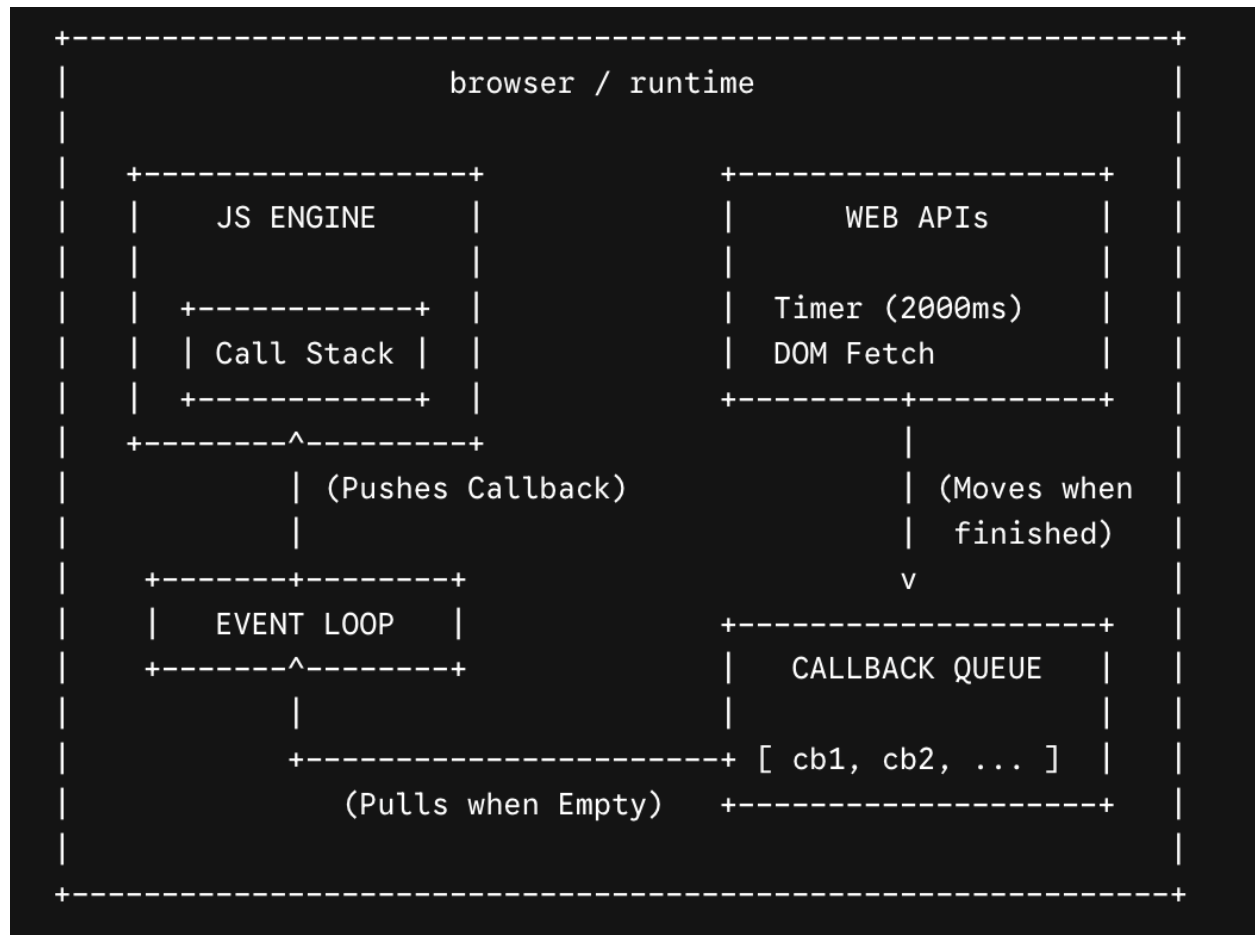
JavaScript Runtime and Execution

JavaScript is often called a "**single-threaded, non-blocking, asynchronous, concurrent**" language.

That sounds like a massive contradiction—how can something be **single-threaded** *and* **concurrent**?

The Architecture of the JavaScript Runtime

Think of the runtime as a bustling factory. The JavaScript Engine is the main worker, but the Runtime provides the conveyor belts, assistant workers, and storage rooms.



A. The JavaScript Engine (e.g., V8 in Chrome/Node.js)

The engine's job is to read and execute your code. It consists of two main parts:

- **The Memory Heap**: A large, mostly unstructured region of memory where the engine stores objects, variables, and functions you define.
- **The Call Stack**: A LIFO (Last In, First Out) data structure that tracks where we are in the program. Whatever function is at the top of the stack is what the engine is currently executing.

B. Web APIs (or C++ APIs in Node.js)

Because the engine has only *one* Call Stack, it can only do one thing at a time. If you ask it to wait 5 seconds, it would freeze your whole browser. To prevent this, browsers give us **Web APIs**. These are external threads handles by the browser, not the JS engine. They include:

- `setTimeout()` and `setInterval()`
- DOM Events (clicks, scrolls, keypresses)
- Fetch / XMLHttpRequest (network requests)

C. The Callback Queue (Task Queue)

When a Web API finishes its job (e.g., a timer expires or a network request returns data), it doesn't just shove its callback function straight into the Call Stack—that would disrupt running code. Instead, it places the callback into the **Callback Queue**, waiting for its turn.

D. The Microtask Queue

A special, high-priority queue specifically for **Promises** (`.then()`, `.catch()`, `async/await`) and `MutationObserver`. It always cuts in front of the regular Callback Queue.

E. The Event Loop

The referee of the whole system. The Event Loop has one simple job: it constantly looks at the **Call Stack** and the **Queues**.

The Golden Rule: If the Call Stack is completely **empty**, the Event Loop takes the first task from the Microtask Queue (or Callback Queue if microtasks are empty) and pushes it onto the Call Stack to be executed.

A Complete Code Example & Step-by-Step Execution

```
console.log("1. Start");

setTimeout(() => {
  console.log("2. Timeout (0ms)");
}, 0);

Promise.resolve().then(() => {
  console.log("3. Promise");
});

console.log("4. End");
```

The Output:

If you run this, the output will always be:

1. 1. Start
2. 4. End
3. 3. Promise
4. 2. Timeout (0ms)

Why? Step-by-Step Execution Tracking:

1. `console.log("1. Start")` enters the **Call Stack**. It prints to the console immediately and pops off the stack.
2. `setTimeout(..., 0)` enters the **Call Stack**.
 - The JS engine recognizes this is a Web API. It hands the timer over to the browser and pops `setTimeout` off the stack.
 - The browser starts a 0ms timer. Because it's 0ms, it finishes *instantly* and moves the inner callback function `() => { console.log("2...") }` into the **Callback Queue**.
3. `Promise.resolve().then(...)` enters the **Call Stack**.
 - The engine recognizes this is a Promise. The callback `() => { console.log("3...") }` is immediately sent to the **Microtask Queue**. The Promise pops off the stack.

4. `console.log("4. End")` enters the **Call Stack**. It prints to the console immediately and pops off the stack.
5. **The Call Stack is now EMPTY.**
6. The **Event Loop** wakes up and checks the queues. It looks at the **Microtask Queue** first.
 - It finds the Promise callback. It pushes it onto the **Call Stack**.
 - 3. Promise is printed. The Call Stack is empty again.
7. The **Event Loop** checks the Microtask Queue again (it's empty), so it checks the **Callback Queue**.
 - It finds the `setTimeout` callback. It pushes it onto the **Call Stack**.
 - 2. Timeout (0ms) is printed.

This is why a 0ms timeout doesn't mean "run immediately"—it means "run as soon as the stack is clear and all microtasks are done!"

Asynchronous Evaluation

The Three Eras of JavaScript Async

Era	Syntax Pattern	The Main Problem It Solved	The New Problem It Created
1. The Past	Callbacks	Allowed code to run non-blocking after a task finished.	"Callback Hell" and unreadable nested code.
2. The Transition	Promises	Escaped nesting by using flat, chainable <code>.then()</code> blocks.	Still required heavy boilerplate and verbose chaining.
3. The Present	Async / Await	Made async code look and behave exactly like synchronous code.	Requires understanding Promises under the hood; easy to accidentally run things sequentially.

Imagine you order a pizza.

1. Callback Era

You tell the restaurant:

"When my pizza is ready, call me."

```
orderPizza(function(pizza) {  
    eatPizza(pizza);  
});
```

The restaurant calls your function later.

Problem: What if after eating, you want dessert, then coffee, then payment confirmation?

The Downside: "Callback Hell" (The Pyramid of Doom)

Callbacks work fine for a single operation. But what if you need to fetch a user, then fetch their orders, then fetch the shipping details for those orders, and then generate an invoice?

You end up with deeply nested code that crawls horizontally across your screen:

```
orderPizza(function(pizza) {  
  eatPizza(pizza, function() {  
    orderDessert(function(dessert) {  
      eatDessert(function() {  
        orderCoffee(function(coffee) {  
          drinkCoffee();  
        });  
      });  
    });  
  });  
});
```

This creates the Pyramid of Doom (Callback Hell).

2. The Promises Era (ES6 / 2015)

To rescue developers from Callback Hell, ES6 introduced **Promises**. A Promise is an object representing the *eventual* completion (or failure) of an asynchronous operation.

A Promise exists in one of three states:

- **Pending:** The operation is still working.
- **Fulfilled:** The operation completed successfully (`.then()` triggers).
- **Rejected:** The operation failed with an error (`.catch()` triggers).

Here is how the nested callback chain looks when refactored into a clean, flat Promise chain:


```

orderPizza()
  .then(eatPizza)
  .then(orderDessert)
  .then(eatDessert)
  .then(orderCoffee)
  .then(drinkCoffee)
  .catch(handleError);

```

While Promises drastically improved readability and error handling, writing long `.then()` chains with multiple anonymous return functions could still become verbose and difficult to debug.

3. The Async / Await Era (ES8 / 2017)

The ultimate evolution of asynchronous JavaScript is `async/await`. It is built completely on top of Promises, acting as "syntactic sugar." It doesn't change how JavaScript runs under the hood; it just makes asynchronous code look like clean, readable, synchronous code.

- **async**: Placed before a function to indicate it will return a Promise.
- **await**: Placed before a Promise. It pauses the execution of the `async` function until the Promise resolves or rejects.

Here is the exact same sequential workflow written in modern `async/await`:

```

async function processInvoice(userId) {
  try {
    // Code looks sequential and pauses cleanly at each 'await'
    const profile = await getProfile(userId);
    const orders = await getOrders(profile.id);
    const shipping = await getShippingDetails(orders[0].id);
    const invoice = await generateInvoice(shipping);

    console.log(invoice);
  } catch (error) {
    // Standard, readable try/catch block handles all async errors
    console.error("An error occurred during processing:", error);
  }
}

processInvoice(101);

```

The Event Loop Connection

Remember: `await` does **not** block the entire browser thread. When the runtime hits an `await`, it pauses *only that specific function*, steps out of it, continues running other scripts on your page, and puts the paused function into the **Microtask Queue** to resume as soon as the awaited Promise finishes.

Notes: Callbacks solved asynchronous execution,
Promises solved callback hell,
Async/Await solved Promise readability issues,
and the Event Loop makes all of them work without blocking JavaScript's single thread.

Microtask and Macrotask

What are Macrotasks?

Macrotasks are the "regular" tasks that the Event Loop processes one at a time.

Examples:

- `setTimeout()`
- `setInterval()`
- DOM Events (click, scroll)
- `setImmediate()` (Node.js)
- I/O operations

None

```
setTimeout(() => {  
  console.log("Macrotask");  
}, 0);
```

This callback goes into the **Macrotask Queue**.

What are Microtasks?

Microtasks are high-priority tasks that run immediately after the current code finishes and **before any macrotask**.

Examples:

- `Promise.then()`
- `Promise.catch()`
- `Promise.finally()`
- `await`
- `queueMicrotask()`
- `MutationObserver`

None

```
Promise.resolve().then(() => {
```

```
    console.log("Microtask");  
  });
```

This callback goes into the **Microtask Queue**.

Event Loop Priority

None

Call Stack

↓

Microtask Queue

↓

Macrotask Queue

The Event Loop always:

1. Executes all synchronous code.
2. Executes all Microtasks.
3. Executes one Macrotask.
4. Repeats.

Example 1

None

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("End");
```

Output

```
None  
Start  
End  
Promise  
Timeout
```

Execution Flow

```
None  
Call Stack:  
Start  
End  
  
Microtask Queue:  
Promise  
  
Macrotask Queue:  
Timeout
```

Event Loop executes:

```
None  
Start  
End  
Promise  
Timeout
```

Example 2 (Multiple Microtasks)

None

```
console.log("1");

Promise.resolve().then(() => {
  console.log("2");
});

Promise.resolve().then(() => {
  console.log("3");
});

setTimeout(() => {
  console.log("4");
}, 0);

console.log("5");
```

Output

None

```
1
5
2
3
4
```

Why?

None

Sync Code:

```
1
5
```

Microtasks:

```
2
3
```

Macrotask:
4

Example 3 (Microtask inside Macrotask)

```
None
setTimeout(() => {
  console.log("A");

  Promise.resolve().then(() => {
    console.log("B");
  });

  console.log("C");
}, 0);
```

Output

```
None
A
C
B
```

Flow

```
None
Macrotask starts
A
Promise added to Microtask Queue
C
Macrotask ends
Microtask executes
B
```

Async/Await and Microtasks

None

```
async function test() {  
  console.log("1");  
  
  await Promise.resolve();  
  
  console.log("2");  
}  
  
test();  
  
console.log("3");
```

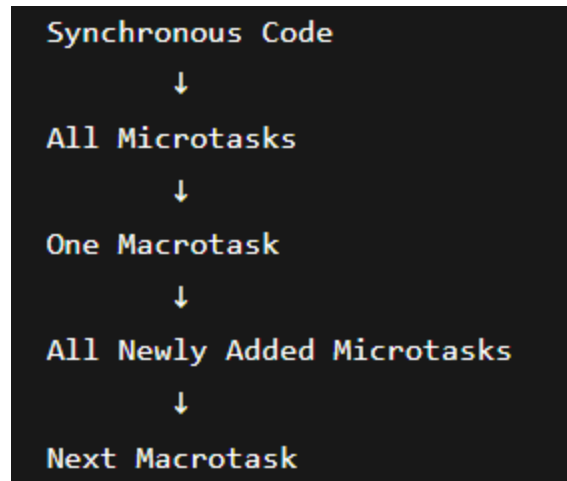
Output

None

1
3
2

Because `await` resumes execution through the **Microtask Queue**.

Easy Rule to Remember:



Promises (then, catch, await) → Microtasks → Higher Priority

setTimeout, setInterval, DOM Events → Macrotasks → Lower Priority

Scope closure and Lexical Environment

1. Scope & The Scope Chain (The Boundary)

While Lexical Environment is the *mechanism*, **Scope** is the *rule*. Scope defines the accessibility and visibility of variables in different parts of your code.

JavaScript has three main types of scope:

- **Global Scope:** Variables declared outside any function or block. Accessible from anywhere.
- **Function Scope:** Variables declared inside a function. They cannot be accessed outside that function.
- **Block Scope (ES6):** Variables declared with `let` or `const` inside a block `{ }` (like an `if` statement or a `for` loop). They are restricted entirely to that block.

The Scope Chain

When you try to access a variable, JavaScript will first look inside the **current scope**. If it can't find it, it travels up the **Scope Chain** (using those outer lexical references) until it finds it or reaches the global scope. If it reaches the global scope and still can't find it, it throws a `ReferenceError`.

2. Closures (The Magic Connection)

A **Closure** happens when a function is defined inside another function, and the inner function *remembers* and *retains access* to its outer lexical environment, **even after the outer function has finished executing**.

In many programming languages, when a function finishes running, its local variables are wiped from memory (garbage collected). In JavaScript, if an inner function is still accessible somewhere (e.g., it was returned or assigned to a global variable), the engine keeps the outer function's Lexical Environment alive in memory.

Code Example: The Counter

```
function createCounter() {  
  let count = 0; // Local variable in the outer scope  
  
  // This inner function is a Closure  
  return function() {  
    count++; // Accesses 'count' from the parent lexical environment  
    console.log(count);  
  };  
}  
  
// 'counter' now holds the inner function returned by createCounter  
const counter = createCounter();  
  
counter(); // Outputs: 1  
counter(); // Outputs: 2  
counter(); // Outputs: 3
```

What just happened?

1. We invoked `createCounter()`, which created a Lexical Environment where `count = 0`.
2. `createCounter()` returned the inner anonymous function and popped off the Execution Call Stack. **Its execution is completely over.**
3. Normally, `count` should be deleted. But because the returned function references `count`, a **Closure** is formed. The inner function preserves a backstage pass to `createCounter`'s Lexical Environment.
4. Every time we call `counter()`, it modifies that same preserved `count` variable.

Homework:

Learn how inheritance is performed in Javascript.

Hoisting and TDZ

Hoisting is JavaScript's default behavior of moving declarations to the top of their containing scope (either the global scope or the current function scope) during the **Memory Creation Phase** before the code is actually executed.

It is crucial to understand that JavaScript only hoists *declarations*, not *initializations* (assignments).

How the JavaScript Engine Sees Your Code

When you run a piece of JavaScript code, the engine splits execution into two phases:

1. **Memory Allocation Phase:** It scans the code for variable and function declarations and sets aside memory for them.
2. **Execution Phase:** It executes the code line-by-line, assigning values and running functions.

```
// What you write:
console.log(hoistedVar); // Outputs: undefined
var hoistedVar = "JavaScript";
console.log(hoistedVar); // Outputs: "JavaScript"

// How the JS Engine interprets it (Hoisting in action):
var hoistedVar;           // Declaration is moved to the top and initialized as undefined
console.log(hoistedVar); // Hence, this logs undefined without throwing an error
hoistedVar = "JavaScript"; // Assignment stays exactly where it was
console.log(hoistedVar); // Logs "JavaScript"
```

Variable Declarations: var vs let vs const

While all variables in JavaScript are technically hoisted, how they behave during hoisting depends entirely on the keyword used to declare them.

1. The var Keyword

- **Scope:** Function-scoped. If declared outside a function, it becomes globally scoped and attaches itself to the global window object. It completely ignores block scopes (like `if` statements or `for` loops).
- **Hoisting:** Hoisted to the top of its scope and automatically initialized with a value of `undefined`.
- **Re-declaration:** You can freely re-declare and re-assign `var` variables within the same scope without errors.


```
if (true) {  
    var test = "I am a var";  
}  
console.log(test); // "I am a var" -> Accessible outside the block
```

2. The let Keyword

- **Scope:** Block-scoped. A block is anything wrapped in curly braces {}. It is only accessible within that specific block.
- **Hoisting:** Hoisted to the top of its block, but **not initialized**. It is placed in a state called the **Temporal Dead Zone (TDZ)**.
- **Re-declaration:** Cannot be re-declared within the same scope, but can be re-assigned.

```
console.log(myLet); // ReferenceError: Cannot access 'myLet' before initialization  
let myLet = "I am a let";
```

3. The const Keyword

- **Scope:** Block-scoped (just like let).
- **Hoisting:** Hoisted to the top of its block, but not initialized (also stays in the Temporal Dead Zone).
- **Re-declaration & Assignment:** Strict limits. It **must be initialized at the time of declaration**, and it cannot be re-assigned or re-declared.

⚠ **Important Caveat:** While you cannot re-assign a const primitive value, const objects and arrays are **mutable**. You can change their internal properties or elements, you just cannot assign a completely new object or array to that variable name.

```
const user = { name: "John" };  
user.name = "Alex"; // Absolutely fine! The object mutated.  
// user = { name: "Bob" }; // TypeError: Assignment to constant variable.
```

What is the Temporal Dead Zone (TDZ)?

The TDZ is the period of time/area of code from the start of the block scope until the moment the JavaScript engine completely processes the `let` or `const` declaration line.

If you try to access a `let` or `const` variable inside this zone, JavaScript will throw a strict `ReferenceError`. This is a safety feature designed to prevent bugs caused by using variables before they have been properly set up.

```
{  
  // === START OF TEMPORAL DEAD ZONE FOR 'data' ===  
  // console.log(data); // Throws ReferenceError  
  
  let data = "Loaded Info"; // === END OF TEMPORAL DEAD ZONE ===  
  console.log(data); // Works perfectly: "Loaded Info"  
}
```

MAP,filter

1. Array.prototype.map()

The `map()` method creates a **brand new array** populated with the results of calling a provided function on every single element in the calling array.

- **Use case:** When you need to transform every item in an array 1-to-1 (e.g., doubling numbers, converting strings to uppercase, extracting specific object properties).
- **Array Length:** The output array is always exactly the same length as the input array.

```
const products = [  
  { name: "Laptop", price: 1000 },  
  { name: "Phone", price: 500 },  
  { name: "Tablet", price: 300 }  
];  
  
// map() takes a callback function: (element, index, array)  
const prices = products.map(product => product.price);  
  
console.log(prices);           // Output: [1000, 500, 300]  
console.log(products);        // Output: (Original array remains completely unchanged!)
```

2. Array.prototype.filter()

The `filter()` method creates a **shallow copy of a portion of a given array**, filtered down to just the elements from the given array that pass a specific test (implemented by the provided callback function).

- **Use case:** When you want to select a subset of items based on a condition (e.g., getting only even numbers, finding active users).
- **The Callback Rule:** Your callback function *must* return a boolean value: `true` to keep the item, or `false` to discard it.
- **Array Length:** The output array can be shorter than or equal to the original array length.

```
const products = [  
  { name: "Laptop", price: 1000 },  
  { name: "Phone", price: 500 },  
  { name: "Tablet", price: 300 }  
];  
  
// The callback returns true if price > 400  
const premiumProducts = products.filter(product => product.price > 400);  
  
console.log(premiumProducts);  
/* Output:  
[  
  { name: "Laptop", price: 1000 },  
  { name: "Phone", price: 500 }  
]  
*/
```

3. Array.prototype.reduce()

The `reduce()` method executes a user-supplied "reducer" callback function on each element of the array, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements of the array is a **single value**.

- **Use case:** When you need to aggregate an array into one final output (e.g., summing up numbers, tallying item frequencies, flattening arrays, compounding an object).
- **Key Components:** It accepts two arguments: the callback function and an **Initial Value** (highly recommended to always provide this!).

The callback function takes four arguments, but the first two are the most critical:

1. **accumulator:** The accumulated value rolled over from the last loop's return statement.
2. **currentValue:** The current item being processed in the loop.

```
const products = [
  { name: "Laptop", price: 1000 },
  { name: "Phone", price: 500 },
  { name: "Tablet", price: 300 }
];

// Syntax: array.reduce((accumulator, currentValue) => { ... }, initialValue)
const totalBill = products.reduce((total, product) => {
  return total + product.price;
}, 0); // 0 is the starting value for 'total'

console.log(totalBill); // Output: 1800
```

Ultimate Example: Find the total value of premium products only

```
const products = [
  { name: "Laptop", price: 1000 },
  { name: "Phone", price: 500 },
  { name: "Tablet", price: 300 }
];

const totalPremiumCost = products
  .filter(p => p.price > 400)      // Step 1: Filters out the Tablet
  .map(p => p.price)               // Step 2: Transforms objects to numbers [1000, 500]
  .reduce((sum, price) => sum + price, 0); // Step 3: Sums them up

console.log(totalPremiumCost); // Output: 1500
```

Questions:

1. https://www.hackerrank.com/challenges/js10-create-a-button/problem?isFullScreen=true&hr_b=1
2. https://www.hackerrank.com/challenges/js10-buttons-container/problem?isFullScreen=true&hr_b=1

JSON Processing

JSON Processing in JavaScript

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data-interchange format used for storing and exchanging data between a client and a server.

Characteristics of JSON

- Human-readable format
- Lightweight and easy to understand
- Language-independent
- Uses key-value pairs
- Commonly used in REST APIs

JSON Example

```
None
{
  "id": 101,
  "name": "aditya",
  "email": "aditya@gmail.com",
  "skills": ["Java", "React", "Node.js"],
  "isActive": true
}
```

JSON Data Types

JSON supports the following data types:

Data Type	Example
String	"John"
Number	100
Boolean	true, false

Array ["Java", "Python"]

Object {"name":"John"}

Null null

JSON vs JavaScript Object

JSON

None

```
{  
  "name": "John",  
  "age": 25  
}
```

JavaScript Object

None

```
const user = {  
  name: "John",  
  age: 25  
};
```

Differences

JSON	JavaScript Object
Keys must be in double quotes	Quotes are optional
Stored as string	Stored as object
Cannot contain functions	Can contain functions

JSON Parsing

What is Parsing?

Converting a JSON string into a JavaScript object.

Syntax

None

```
JSON.parse(jsonString);
```

Example

None

```
const jsonData = '{"name":"Rajalaxmi","age":24}';
```

```
const user = JSON.parse(jsonData);
```

```
console.log(user.name);
```

```
console.log(user.age);
```

Output

None

```
Rajalaxmi
```

```
24
```

JSON Stringification

What is Stringification?

Converting a JavaScript object into a JSON string.

Syntax

None

```
JSON.stringify(object);
```

Example

None

```
const student = {  
  id: 101,  
  name: "Rajalaxmi",  
  course: "MCA"  
};  
  
const jsonString = JSON.stringify(student);  
  
console.log(jsonString);
```

Output

None

```
{"id":101,"name":"Rajalaxmi","course":"MCA"}
```

Accessing JSON Data

Using Dot Notation

None

```
const employee = {  
  name: "Rahul",  
  salary: 50000  
};  
  
console.log(employee.name);
```

Using Bracket Notation

None

```
console.log(employee["salary"]);
```

Nested JSON Objects

None

```
const company = {  
  name: "ABC Ltd",  
  employee: {  
    id: 101,  
    name: "John"  
  }  
};  
  
console.log(company.employee.name);
```

Output

None

John

JSON Arrays

None

```
const students = [  
  {  
    id: 1,  
    name: "John"  
  },  
  {  
    id: 2,  
    name: "bob"  
  }  
];  
  
console.log(students[0].name);
```

Output

None

bob

Iterating Through JSON Data

Using for...of

None

```
const students = [  
  { id: 1, name: "John" },  
  { id: 2, name: "bob" }  
];  
  
for (const student of students) {  
  console.log(student.name);  
}
```

Output

None

John
bob

Fetching JSON from API

None

```
fetch('https://jsonplaceholder.typicode.com/users')  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {
```

```
        console.log(error);  
    });
```

Async/Await Version

None

```
async function getUsers() {  
    try {  
        const response = await fetch(  
            'https://jsonplaceholder.typicode.com/users'  
        );  
  
        const data = await response.json();  
  
        console.log(data);  
    }  
    catch(error) {  
        console.error(error);  
    }  
}  
  
getUsers();
```

JSON Processing Operations

Reading JSON

None

```
const jsonData = '{"name":"John"}';  
  
const user = JSON.parse(jsonData);  
  
console.log(user.name);
```

Updating JSON Data

None

```
const user = {  
  name: "John",  
  age: 25  
};  
  
user.age = 30;  
  
console.log(user);
```

Adding New Property

None

```
const user = {  
  name: "John"  
};  
  
user.city = "Delhi";  
  
console.log(user);
```

Deleting Property

None

```
const user = {  
  name: "John",  
  age: 25  
};  
  
delete user.age;
```



```
console.log(user);
```

Common JSON Methods

Method	Description
JSON.parse()	Converts JSON string to object
JSON.stringify()	Converts object to JSON string

DOM and CSSOM

1. What is DOM (Document Object Model)?

The **DOM** is a programming interface that represents an HTML document as a tree of objects. It allows JavaScript to access, modify, add, or delete HTML elements dynamically.

Example HTML

None

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Example</title>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <button onclick="changeText()">Click Me</button>

  <script>
    function changeText() {
      document.getElementById("title").innerHTML = "DOM
Updated!";
    }
  </script>
</body>
</html>
```

DOM Tree Structure

None

```
Document
|
├─ html
│   ├── head
│   │   └─ title
│   └─ body
│       └─ h1
```

└─ button

Common DOM Methods

Method	Purpose
<code>getElementById()</code>	Select element by ID
<code>getElementsByName()</code>	Select elements by class
<code>getElementsByTagName()</code>	Select elements by tag
<code>querySelector()</code>	Select first matching element
<code>querySelectorAll()</code>	Select all matching elements
<code>createElement()</code>	Create a new element
<code>appendChild()</code>	Add an element
<code>removeChild()</code>	Remove an element

DOM Manipulation Example

```
None
const para = document.createElement("p");
para.innerText = "New Paragraph";
document.body.appendChild(para);
```

2. What is CSSOM (CSS Object Model)?

The **CSSOM** is the browser's representation of all CSS styles applied to a webpage.

Just as the DOM represents HTML elements, the CSSOM represents CSS rules and styles.

Example CSS

None

```
h1 {  
  color: blue;  
  font-size: 30px;  
}
```

CSSOM Structure

None

```
StyleSheet  
├──  
└─ CSSRule  
    ├── Selector: h1  
    ├── color: blue  
    └─ font-size: 30px
```

Accessing CSSOM with JavaScript

None

```
const heading = document.querySelector("h1");  
  
console.log(heading.style.color);
```

Modifying Styles

None

```
const heading = document.getElementById("title");  
  
heading.style.color = "red";  
heading.style.fontSize = "40px";
```

Output:

None

```
color: red;  
font-size: 40px;
```

DOM vs CSSOM

Feature	DOM	CSSOM
Represents	HTML Structure	CSS Styles
Built From	HTML Document	CSS Rules
Used For	Content Manipulation	Style Manipulation
Accessible Through	document object	styleSheets object
Example	Add/Delete Elements	Change Colors, Fonts

How Browser Uses DOM and CSSOM

When a webpage loads:

Step 1: Parse HTML

Browser creates the **DOM Tree**.

None

```
<h1>Hello</h1>
```



None

```
Document
```

```
└─ h1
```

Step 2: Parse CSS

Browser creates the **CSSOM Tree**.

None

```
h1 {
```

```
color: blue;
}
```



```
None
StyleSheet
└─ h1 { color: blue }
```

Step 3: Create Render Tree

Browser combines DOM + CSSOM.

```
None
DOM + CSSOM
  ↓
Render Tree
```

Step 4: Layout

Browser calculates:

- Width
- Height
- Position

Step 5: Paint

Browser draws pixels on the screen.

```
None
HTML
  ↓
DOM
  ↓
CSS
  ↓
```

CSSOM

↓

Render Tree

↓

Layout

↓

Paint

Real-Time Example

None

```
<h1 id="heading">JavaScript</h1>
```

```
<script>
```

```
const heading = document.getElementById("heading");
```

```
// DOM Manipulation
```

```
heading.innerText = "DOM Changed";
```

```
// CSSOM Manipulation
```

```
heading.style.color = "green";
```

```
heading.style.backgroundColor = "yellow";
```

```
</script>
```

Result

- DOM changes the content:

None

JavaScript → DOM Changed

- CSSOM changes the appearance:

None

```
color: green;  
background-color: yellow;
```

Interview Question

Q: What is the difference between DOM and CSSOM?

Answer:

The DOM represents the structure and content of an HTML document, while the CSSOM represents the styles applied to that document. The browser combines both to create the Render Tree, which is used to display the webpage on the screen. DOM is manipulated using the document object, whereas CSSOM is manipulated through style-related APIs such as `element.style` and `document.styleSheets`.

ES6

ECMAScript 6 (ES6) – Modern JavaScript

1. Introduction to ES6

ECMAScript 6 (ES6), also known as ECMAScript 2015, is a major update to JavaScript that introduced several modern features to make code more readable, maintainable, and efficient.

Why ES6?

- Cleaner and shorter syntax
- Better code organization
- Improved support for Object-Oriented Programming
- Easier asynchronous programming
- Enhanced performance and maintainability

ES6 Features

|

|— **let & const**

|— **Arrow Functions**

|— **Template Literals**

|— **Default Parameters**

|— **Destructuring**

|— **Spread Operator**

|— **Rest Parameter**

|— **Classes**

|— **Inheritance**

|— **Modules**

|— **Promises**

|— **Symbol**

|— **Map**

|— **Set**

|— **for...of**

2. let and const

ES6 introduced two new ways to declare variables: `let` and `const`.

let

A block-scoped variable that can be reassigned.

Example

JavaScript

```
let age = 25;  
age = 30;  
console.log(age);
```

Features

- Block scoped
- Can be updated
- Cannot be redeclared in the same scope

const

A block-scoped variable whose reference cannot be reassigned.

Example

JavaScript

```
const PI = 3.14;
```

Features

- Block scoped
- Cannot be reassigned
- Must be initialized during declaration

Difference Between var, let, and const

Feature	var	let	const
Scope	Function	Block	Block
Reassignment	Yes	Yes	No
Redeclaration	Yes	No	No
Hoisting	Yes	Yes	Yes

3. Arrow Functions

Arrow functions provide a concise syntax for writing functions.

Traditional Function

```
JavaScript
function add(a, b) {
  return a + b;
}
```

Arrow Function

```
JavaScript
const add = (a, b) => a + b;
```

Benefits

- Less code
- Improved readability
- Lexical binding of this

4. Template Literals

Template literals allow embedding expressions inside strings using backticks.

Example

```
JavaScript
let name = "Raj";
console.log(`Welcome ${name}`);
```

Advantages

- String interpolation
- Multi-line strings
- Improved readability

5. Default Parameters

Default values can be assigned directly in function parameters.

Example

```
JavaScript
function greet(name = "Guest") {
    console.log(`Hello ${name}`);
}

greet();
```

Output

```
None
Hello Guest
```

6. Destructuring

Destructuring extracts values from arrays and objects into separate variables.

Array Destructuring

JavaScript

```
const colors = ["Red", "Blue", "Green"];

const [first, second] = colors;
```

Object Destructuring

JavaScript

```
const student = {
  name: "Raj",
  age: 22
};

const { name, age } = student;
```

Benefits

- Cleaner code
- Easier extraction of values
- Reduces repetition

7. Spread Operator (...)

Used to expand arrays or objects.

Array Example

JavaScript

```
const arr1 = [1, 2, 3];
```



```
const arr2 = [...arr1, 4, 5];
```

Object Example

JavaScript

```
const user = {  
  name: "Raj"  
};  
  
const updatedUser = {  
  ...user,  
  city: "Delhi"  
};
```

Uses

- Copy arrays
 - Merge arrays
 - Clone objects
 - Merge objects
-

8. Rest Parameters (...)

Used to collect multiple arguments into a single array.

Example

JavaScript

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}
```

Output

JavaScript

```
sum(10, 20, 30);
```

Result:

None

60

9. Enhanced Object Literals

Simplified syntax for creating objects.

Example

JavaScript

```
let name = "Raj";

const user = {
  name
};
```

Benefit

Reduces code duplication.

10. Classes

ES6 introduced class-based syntax for Object-Oriented Programming.

Example

JavaScript

```
class Student {
```

```
    constructor(name) {  
        this.name = name;  
    }  
  
    display() {  
        console.log(this.name);  
    }  
}
```

Creating an Object

JavaScript

```
const s1 = new Student("Raj");  
s1.display();
```

11. Inheritance

Classes can inherit properties and methods from other classes.

Example

JavaScript

```
class Animal {  
    speak() {  
        console.log("Animal Sound");  
    }  
}  
  
class Dog extends Animal {  
    bark() {  
        console.log("Woof Woof");  
    }  
}
```

Benefit

Promotes code reusability.

12. Modules

Modules allow splitting code into multiple files.

Export

JavaScript

```
export const name = "Raj";
```

Import

JavaScript

```
import { name } from "../file.js";
```

Advantages

- Better project organization
 - Code reusability
 - Easier maintenance
-

13. Promises

Promises handle asynchronous operations.

States of a Promise

1. Pending
2. Fulfilled
3. Rejected

Example

JavaScript

```
const promise = new Promise((resolve, reject) => {

    let success = true;

    if(success) {
        resolve("Success");
    } else {
        reject("Failed");
    }
});
```

Handling Promise

JavaScript

```
promise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

14. Map

Map stores key-value pairs and maintains insertion order.

Example

JavaScript

```
const map = new Map();

map.set("name", "Raj");
map.set("age", 22);

console.log(map.get("name"));
```

Advantages

- Any data type can be used as keys
 - Better performance for frequent additions/removals
-

15. Set

A Set stores unique values only.

Example

JavaScript

```
const numbers = new Set();  
  
numbers.add(10);  
numbers.add(10);  
numbers.add(20);
```

Output

None

```
{10, 20}
```

Benefits

- Removes duplicates
 - Efficient lookups
-

16. for...of Loop

Used to iterate over iterable objects.

Example

JavaScript

```
const arr = [10, 20, 30];
```

```
for(const value of arr) {  
  console.log(value);  
}
```

Output

```
None  
10  
20  
30
```

Advantages of ES6

- Cleaner and modern syntax
 - Better code readability
 - Improved maintainability
 - Supports modular development
 - Simplifies asynchronous programming
 - Encourages Object-Oriented Programming
 - Widely used in React, Angular, Vue, and Node.js
-

Interview Questions

1. What is ES6?

ES6 is the sixth version of ECMAScript that introduced modern JavaScript features to improve development efficiency and code quality.

2. What is the difference between let and const?

- let can be reassigned.
- const cannot be reassigned after initialization.

3. What are Arrow Functions?

Arrow functions provide a shorter syntax for writing functions and automatically inherit the surrounding `this` value.

4. What is Destructuring?

Destructuring is a technique used to extract values from arrays or objects into individual variables.

5. Difference Between Rest and Spread Operator?

- Rest Operator collects values.
- Spread Operator expands values.

6. What are Promises?

Promises represent the eventual completion or failure of an asynchronous operation.

7. What are Modules?

Modules allow code to be split into reusable files using `import` and `export` statements.

8. What is the purpose of Map and Set?

- Map stores key-value pairs.
- Set stores unique values only.

Event Handling , Hooks

Event Handling in React

What is an Event?

An **event** is an action performed by the user or browser.

Examples:

- Clicking a button
 - Typing in an input field
 - Hovering over an element
 - Submitting a form
-

What is Event Handling?

Definition:

Event handling is the process of executing a function when a particular event occurs.

Example:

```
None
function App() {

  function handleClick() {
    alert("Button Clicked!");
  }

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

```
}
```

React vs HTML Event Handling

HTML

None

```
<button onclick="showMessage()">  
  Click  
</button>
```

React

None

```
<button onClick={showMessage}>  
  Click  
</button>
```

Notice:

- `onclick` → `onClick`
- Pass function reference, not function call.

Common React Events

Event	Description
<code>onClick</code>	Mouse click
<code>onChange</code>	Input changes
<code>onSubmit</code>	Form submission
<code>onMouseOver</code>	Mouse hover

onKeyDown	Key pressed
onFocus	Input focused
onBlur	Input loses focus

Synthetic Events

Definition

A **Synthetic Event** is a wrapper around the browser's native event.

React creates its own event system to make events work consistently across all browsers.

```
None
Browser Event
  ↓
React Synthetic Event
  ↓
Your Event Handler
```

Example

```
None
function App() {

  function handleClick(event) {
    console.log(event);
  }

  return (
    <button onClick={handleClick}>
      Click
    </button>
  );
}
```

```
}
```

event here is not the native browser event.

It is a **SyntheticEvent** object.

Benefits of Synthetic Events

- Cross-browser compatibility
 - Better performance
 - Same API everywhere
-

Passing Arguments to Event Handlers

Suppose we want to send data.

Wrong

None

```
<button onClick={showMessage("React")}>
```

This executes immediately during rendering.

Correct

None

```
<button onClick={() => showMessage("React")}>  
  Click  
</button>
```

None

```
function showMessage(course){  
  alert(course);  
}
```

Output:

None

React

Passing Event and Argument Together

None

```
function showMessage(event, name){  
  console.log(event);  
  console.log(name);  
}  
  
<button  
  onClick={(e)=>showMessage(e, "Rajalaxmi")}  
>  
  Click  
</button>
```

Event Bubbling

Definition

Event Bubbling means an event starts from the target element and moves upward through its parent elements.

Example:

None

GrandParent

↑

Parent

↑

Child(Button)

Click Button →

Button executes

Then Parent executes

Then GrandParent executes

Example

None

```
function App(){  
  
  return(  
    <div onClick={()=>console.log("Div Clicked")}>  
  
    <button onClick={()=>  
      console.log("Button Clicked")  
    }>  
    Click  
    </button>  
  
    </div>  
  )  
  
}
```

Output:

None

Button Clicked

Div Clicked

Stopping Event Bubbling

None

```
function handleClick(e){  
  e.stopPropagation();  
  console.log("Button");  
}
```

```
<div onClick={()=>console.log("Div")}>
```

```
<button onClick={handleClick}>
```

```
Click
```

```
</button>
```

```
</div>
```

Output:

None

Button

7. Conditional Rendering

Definition

Conditional Rendering means displaying different UI based on certain conditions.

if/else

None

```
function App(){  
  
  let isLoggedIn=true;  
  
  if(isLoggedIn){  
    return <h1>Welcome User</h1>;  
  }  
  else{  
    return <h1>Please Login</h1>;  
  }  
  
}
```

Ternary Operator

Syntax:

None

```
condition ? truePart : falsePart
```

Example:

None

```
function App(){  
  
  const isLoggedIn=true;  
  
  return(  
    <h1>  
    {  
      isLoggedIn ? "Welcome User" : "Please Login"  
    }  
  )  
}
```

```
}  
</h1>  
)  
  
}
```

&& Operator

Used when only one condition needs checking.

```
None  
function App(){  
  
  const isAdmin=true;  
  
  return(  
    <>  
    {isAdmin && <h1>Admin Panel</h1>}  
    </>  
  )  
  
}
```

If true:

```
None  
Admin Panel
```

If false:

Nothing renders.

Switch Case

None

```
function App(){  
  
  const role="admin";  
  
  switch(role){  
  
    case "admin":  
      return <h1>Admin Dashboard</h1>;  
  
    case "user":  
      return <h1>User Dashboard</h1>;  
  
    default:  
      return <h1>Guest</h1>;  
  
  }  
  
}
```

Interview Question

Which conditional rendering method is most common?

Answer:

Ternary operator and && operator.

8. Lists and Keys

Why Lists?

Suppose you want to display students.

```
None  
Rahul  
Amit  
Neha  
Priya
```

Instead of writing multiple `<li>` tags manually, React uses `map()`.

map()

Definition

`map()` creates a new array by applying a function to every element.

JavaScript Example:

```
None  
const arr=[1,2,3];  
  
const result=arr.map(num=>num*2);  
  
console.log(result);
```

Output:

```
None  
[2,4,6]
```

Keys

Definition

A Key is a special attribute that uniquely identifies elements in a list.

None

```
students.map(student=>
```

```
<li key={student}>
```

```
{student}
```

```
</li>
```

```
)
```

Why not use Array Index?

None

```
students.map((student, index)=>
```

```
<li key={index}>
```

```
{student}
```

```
</li>
```

```
)
```

Suppose:

None

```
0 Rahul
```

```
1 Amit
```

```
2 Neha
```

Insert Priya at beginning.

None

0 Priya

1 Rahul

2 Amit

3 Neha

All indexes change.

React thinks every item changed.

This causes:

- Unnecessary re-render
- Performance issues
- State bugs

Best Practice

Use unique id.

None

```
const students=[
  {id:1,name:"Rahul"},
  {id:2,name:"Amit"}
];

students.map(student=>

<li key={student.id}>
  {student.name}
</li>

)
```

Interview Question

Why shouldn't we use array index as key?

Because inserting, deleting, or reordering elements changes indexes, causing React to incorrectly identify elements and perform unnecessary re-renders.

9. Forms

What is a Form?

A form is used to collect user input.

Example:

- Login Form
 - Registration Form
 - Contact Form
-

Controlled Components

Definition

A Controlled Component is a form element whose value is controlled by React State.

None

```
import {useState} from "react";

function App(){

  const [name, setName]=useState("");

  return(

    <>
    <input
```

```
type="text"

value={name}

onChange={(e)=>
  setName(e.target.value)
}

/>

<h2>{name}</h2>

</>

)

}
```

Flow:

```
None
Input
  ↓
onChange
  ↓
State Updates
  ↓
UI Re-renders
```

Uncontrolled Components

Definition

The DOM itself manages the form data.

React accesses it using useRef.

None

```
import {useRef} from "react";

function App(){

  const inputRef=useRef();

  function showData(){
    alert(inputRef.current.value);
  }

  return(
    <>
    <input ref={inputRef}/>

    <button onClick={showData}>
    Submit
    </button>
    </>
  )

}
```

Controlled vs Uncontrolled

Controlled	Uncontrolled
React controls data	DOM controls data
Uses useState	Uses useRef
Easy validation	Less control

Recommended

Rarely used

Form Validation

Definition

Validation ensures user input is correct.

Example:

- Name required
 - Password minimum 8 characters
 - Email format correct
-

Simple Validation

None

```
import {useState} from "react";

function App(){

  const [email,setEmail]=useState("");

  function submitHandler(){

    if(email==="){
      alert("Email Required");
    }

  }

  return(

    <>
```

```
<input  
  
value={email}  
  
onChange={(e)=>  
  setEmail(e.target.value)  
}  
  
/>  
  
<button onClick={submitHandler}>  
  Submit  
</button>  
  
</>  
  
)  
  
}
```

10. React Hooks

What are Hooks?

Definition:

Hooks are special built-in functions that allow Functional Components to use React features like state and lifecycle methods.

Before Hooks:

Only Class Components could use state.

After Hooks (React 16.8):

Functional Components can use state and lifecycle.

Why Hooks were introduced?

Class Components had problems:

- Large boilerplate
- Complex lifecycle methods
- Difficult to reuse logic
- Confusing this keyword

Hooks solve these problems.

Common Hooks

Hook	Purpose
useState	Manage state
useEffect	Side effects
useRef	DOM reference
useMemo	Cache values
useCallback	Cache functions
useContext	Access Context
useReducer	Complex state management

1. useState()

Definition

useState is a Hook used to add and manage state inside Functional Components.

Syntax

None

```
const [state, setState] = useState(initialValue);
```

- state → current value
 - setState → updates state
-

Example

None

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (
    <>
      <h1>{count}</h1>

      <button
        onClick={() => setCount(count + 1)}
      >
        Increment
      </button>
    </>
  );
}
```

When to use?

- Counter
 - Toggle button
 - Form input
 - Shopping cart quantity
-

2. useEffect()

Definition

useEffect is used for performing side effects inside components.

What are Side Effects?

Operations outside rendering:

- API Calls
 - Timers
 - Event Listeners
 - DOM updates
 - Local Storage
-

Syntax

None

```
useEffect(() => {  
  
}, []);
```

Case 1 : Runs Every Render

None

```
useEffect(() => {  
  console.log("Runs");  
});
```

Case 2 : Runs Once

None

```
useEffect(() => {
```

```
    console.log("Only Once");  
  }, []);
```

Case 3 : Runs on Dependency Change

```
None  
useEffect(() => {  
  console.log("Count Changed");  
}, [count]);
```

Cleanup Function

```
None  
useEffect(() => {  
  
  const timer = setInterval(() => {  
    console.log("Running");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
  };  
  
}, []);
```

When to use?

- Fetch API
 - Authentication check
 - Timers
 - Window events
-

3. useRef()

Definition

useRef creates a mutable reference that persists across renders without causing re-rendering.

DOM Access

None

```
import { useRef } from "react";

function App() {

  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <>
      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>
    </>
  );
}
```

Store Previous Value

None

```
const previous = useRef(0);
```



```
previous.current = count;
```

When to use?

- Input focus
 - Stopwatch
 - Previous value tracking
 - Timer IDs
-

4. useMemo()

Definition

useMemo caches (memorizes) expensive calculations and recomputes them only when dependencies change.

Syntax

None

```
const value = useMemo(() => {  
  return expensiveCalculation();  
}, [dependency]);
```

Example

None

```
import { useMemo, useState } from "react";  
  
function App() {
```

```
const [count, setCount] = useState(0);

const square = useMemo(() => {
  console.log("Calculating...");
  return count * count;
}, [count]);

return (
  <>
    <h1>{square}</h1>

    <button
      onClick={() => setCount(count + 1)}
    >
      +
    </button>
  </>
);
}
```

When to use?

- Large array filtering
 - Heavy calculations
 - Expensive computations
-

5. useCallback()

Definition

useCallback memorizes a function so that it is not recreated on every render.

Syntax

None

```
const memoizedFunction =  
useCallback(() => {  
  
}, [dependency]);
```

Example

None

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

When to use?

- Passing functions to child components
 - Working with React.memo
 - Performance optimization
-

useMemo vs useCallback

useMemo	useCallback
Caches values	Caches functions
Returns result	Returns function

6. useContext()

Definition

useContext provides access to shared data without passing props manually.

It solves **Props Drilling**.

Without Context

```
None
App
↓
Parent
↓
Child
↓
GrandChild
```

Passing props through every level.

Example

Create Context

```
None
import { createContext } from "react";

export const UserContext =
  createContext();
```

Provide Context

```
None
<UserContext.Provider
  value={"Rajalaxmi"}>

  <Child/>
```

```
</UserContext.Provider>
```

Consume Context

```
None  
import { useContext } from "react";  
  
const user =  
  useContext(UserContext);  
  
return <h1>{user}</h1>;
```

When to use?

- Theme
 - Login user
 - Language selection
 - Global settings
-

7. useReducer()

Definition

useReducer is used for managing complex state logic.

Syntax

```
None  
const [state, dispatch] =  
  useReducer(reducer, initialState);
```

Example

None

```
import { useReducer } from "react";

function reducer(state, action) {

  switch (action.type) {

    case "increment":
      return state + 1;

    case "decrement":
      return state - 1;

    default:
      return state;
  }
}

function App() {

  const [count, dispatch] =
    useReducer(reducer, 0);

  return (
    <>
      <h1>{count}</h1>

      <button
        onClick={() =>
          dispatch({
            type:"increment"
          })
        }
      >
        +
      </button>
    </>
  )
}
```

```
        </button>
      </>
    );
  }
}
```

When to use?

- Shopping cart
 - Banking system
 - Multiple state transitions
-

8. `useLayoutEffect()`

Definition

Works like `useEffect` but executes before the browser paints the screen.

```
None
Render
  ↓
useLayoutEffect
  ↓
Paint Screen
```

Example

```
None
useLayoutEffect(() => {
  console.log("Runs first");
}, []);
```

When to use?

- Measuring DOM size
 - Preventing flicker
 - Animations
-

useEffect vs useLayoutEffect

useEffect	useLayoutEffect
After paint	Before paint
Async	Sync
Most common	Rare

9. useImperativeHandle()

Definition

Allows a parent component to customize the instance value exposed by a child using refs.

Example

```
None
useImperativeHandle(ref, () => ({
  focus() {
    inputRef.current.focus();
  }
}));
```

When to use?

- Custom input components

- Modal controls
-

10. useId()

Definition

Generates unique IDs for accessibility.

Example

```
None
import { useId } from "react";

function App(){

  const id = useId();

  return(
    <>
    <label htmlFor={id}>
      Name
    </label>

    <input id={id}/>
  </>
  )

}
```

When to use?

- Forms
- Labels
- Accessibility

11. useDeferredValue()

Definition

Delays updating non-critical UI to improve responsiveness.

Example

```
None  
const deferredSearch =  
useDeferredValue(searchText);
```

When to use?

- Search bars
 - Large filtering
 - Autocomplete
-

12. useTransition()

Definition

Marks some state updates as low priority.

Example

```
None  
const [isPending, startTransition]  
= useTransition();
```

```
startTransition(() => {  
  setList(newList);  
});
```

When to use?

- Large UI updates
 - Search filtering
 - Pagination
-

13. useSyncExternalStore()

Definition

Used for subscribing to external stores safely.

Example:

- Redux
 - Zustand
 - Browser APIs
-

Syntax

None

```
const state =  
useSyncExternalStore(  
  subscribe,  
  getSnapshot  
);
```

14. useInsertionEffect()

Definition

Runs before DOM mutations.

Mostly used by CSS-in-JS libraries.

Example:

- Emotion
- Styled Components

Rarely used in normal applications.

Custom Hooks

Definition

A Custom Hook is a JavaScript function whose name starts with "use" and can use other Hooks.

Example

```
None
import { useState } from "react";

function useCounter() {

  const [count, setCount] =
    useState(0);

  const increment = () =>
    setCount(count + 1);

  return {
```

```
    count,  
    increment  
  };  
}
```

Usage:

```
None  
const { count, increment }  
= useCounter();
```

One-line Revision Table

Hook	Purpose
useState	Manage state
useEffect	Side effects
useRef	DOM access / persistent values
useMemo	Cache expensive values
useCallback	Cache functions
useContext	Global data sharing
useReducer	Complex state management
useLayoutEffect	DOM measurement before paint
useImperativeHandle	Custom ref exposure
useId	Unique IDs
useDeferredValue	Delay non-critical updates
useTransition	Low-priority rendering

useSyncExternalStore

External store subscription

useInsertionEffect

CSS injection optimization

Custom Hook

Reusable Hook logic

React Interview Questions

React Basics

1. What is React?
2. What are the features of React?
3. What is a Single Page Application (SPA)?
4. What is Virtual DOM?
5. How does Virtual DOM improve performance?
6. What is JSX?
7. Why do we use JSX?
8. Can React work without JSX?
9. What is Babel?
10. What is the difference between React and JavaScript?

Components

11. What is a component in React?
12. What are Functional Components?
13. What are Class Components?
14. Difference between Functional and Class Components?
15. What is component reusability?
16. What is component composition?
17. What is a Pure Component?
18. What is React Fragment?
19. Why use React Fragments?
20. Difference between Fragment and div?

Props

21. What are props?
22. Why are props read-only?
23. What is props drilling?
24. How do you pass data from parent to child?
25. Can child modify props?
26. What is defaultProps?
27. What is props.children?
28. What is the spread operator in props?
29. How do you pass functions as props?
30. What is prop validation?

State

31. What is state?
32. Difference between state and props?

- 33. Why is state mutable?
- 34. How do you update state?
- 35. What happens when state changes?

Hooks

- 36. What are React Hooks?
- 37. Why were Hooks introduced?
- 38. What is useState?
- 39. How does useState work internally?
- 40. What is useEffect?
- 41. When does useEffect run?
- 42. What is the dependency array?
- 43. Difference between useEffect and useLayoutEffect?
- 44. What is useRef?
- 45. Why use useRef instead of state?
- 46. What is useMemo?
- 47. Why use useMemo?
- 48. What is useCallback?
- 49. Difference between useMemo and useCallback?
- 50. What is useContext?
- 51. What problem does Context API solve?
- 52. What is useReducer?
- 53. Difference between useState and useReducer?
- 54. What is a custom hook?
- 55. How do you create a custom hook?

Event Handling

- 56. How does event handling work in React?
- 57. What is Synthetic Event?
- 58. Difference between Synthetic Event and DOM Event?
- 59. How do you pass arguments to event handlers?
- 60. What is event delegation?

Forms

- 61. What is a controlled component?
- 62. What is an uncontrolled component?
- 63. Difference between controlled and uncontrolled forms?
- 64. How do you handle form submission?
- 65. How do you validate forms in React?
- 66. What is React Hook Form?
- 67. What is Formik?

- 68. Why use React Hook Form?
- 69. How do you reset a form?
- 70. How do you manage multiple inputs?

Routing

- 71. What is React Router?
- 72. What is BrowserRouter?
- 73. Difference between BrowserRouter and HashRouter?
- 74. What is Route?
- 75. What is Routes?
- 76. What is Link?
- 77. Difference between Link and anchor tag?
- 78. What is useNavigate?
- 79. What is useParams?
- 80. What are nested routes?

Performance Optimization

- 81. What is React.memo?
- 82. How does React.memo work?
- 83. When should React.memo be used?
- 84. What causes unnecessary re-renders?
- 85. How do you optimize React applications?
- 86. What is lazy loading?
- 87. What is React.lazy()?
- 88. What is Suspense?
- 89. What is code splitting?
- 90. Difference between client-side rendering and server-side rendering?

Context API & State Management

- 91. What is Context API?
- 92. How does Context API work?
- 93. What are Providers and Consumers?
- 94. Difference between Context API and Redux?
- 95. What is Redux?
- 96. What is Redux Toolkit?
- 97. What is a reducer?
- 98. What is middleware in Redux?
- 99. What is Redux Thunk?
- 100. What is Redux Saga?